Coverage Overlap, Deleted Behaviour and Redundant Tests: An Empirical Check of Test-Suite Audit Heuristics on Eight Python Projects

Tim Richardson
https://github.com/timini/test-suite-auditor

October 2026

Abstract

Test-suite maintenance tools lean on cheap signals: tests that cover the same lines look redundant, a function whose deletion breaks a test looks protected, and a test that passed a few reruns looks stable. Before building these signals into the Test Suite Auditor, a Claude Code plugin, we tested six pre-registered hypotheses about them on eight mature Python projects (8,879 in-scope tests), using the plugin's own runner. Among test pairs with coverage Jaccard of at least 0.9, 35.0% (95% CI 22.2%–47.5%) are complementary: each kills a mutant the other misses. Of the call deletions that no test noticed, 71.4% (58.0%–80.3%) sit inside functions whose every whole-body mutant was killed, a stronger version of a result previously reported for Java. Removing all individually redundant tests from a file together lost kills in 16 of 16 files. Per-test coverage contexts slowed suites by a pooled factor of 2.48, and the only intermittent tests we observed in 60 runs per project were order-dependent: in file order one fails every time and the other never fails. Under equal budgets, deletion mutants yielded 0.38 times as many activated survivors per CPU-minute as mutmut's classical operators (95% CI 0.24–0.48). We release the instrument, raw data and analysis, and describe how each result changes the plugin's design.

1 Introduction

Tools that help maintain a test suite tend to rely on a few cheap signals. Two tests that execute the same lines look redundant. A test that has not failed in a few runs looks stable. A function whose deletion breaks a test looks protected. Each signal is quick to compute, and each can be wrong in ways that matter if a tool turns it into advice to delete or skip tests.

We are building the Test Suite Auditor, a Claude Code plugin that audits a pytest suite and reports evidence about cost, intermittent failures, coverage overlap and fault detection. Its product requirements [12] commit to several rules drawn from published work, most of it on Java. Overlap must never be reported as redundancy. A killed whole-function mutant must not be taken to protect the function's individual effects. Tests that are each removable on their own must be validated as a group before any combined removal. Per-test coverage must be measured separately from runtime profiling because it adds overhead. A short run without failures must be reported as "not observed" rather than "stable".

This study tests whether those rules hold on mature open-source Python projects, before the plugin depends on them. We pre-registered six hypotheses (H1 to H6), pinned eight projects at release commits, and ran every experiment with the same instrument the plugin uses. The hypotheses, metrics and decision rules were committed to the repository before data collection; deviations are listed in the pre-registration file and summarised in Section 5.

The contributions are:

- a replication on Python of the statement-level pseudo-testedness result previously reported for Java [8] (H3);
- measurements of how often high coverage overlap and leave-one-out redundancy mislead on real suites (H1, H4);
- a cost comparison of deterministic deletion mutants against mutmut's classical operators under equal budgets (H2);
- measured overhead of per-test coverage contexts (H5) and a repeat-run flakiness survey (H6);
- the raw data, analysis scripts and instrument, released with the plugin.

2 Background

Coverage and effectiveness. Line coverage measures what a test executes, not what it checks. Inozemtseva and Holmes [5] found that coverage is not strongly correlated with suite effectiveness once suite size is controlled. Mutation analysis is the usual proxy for fault detection: a mutant is a small, deliberate change, and a suite "kills" it if some test fails. Just et al. [6] showed mutant detection correlates with real-fault detection, while Papadakis et al. [10] survey the many threats to interpreting mutation scores, including equivalent mutants and sampling.

Pseudo-tested code. Niedermayr et al. [9] introduced pseudo-tested methods: methods whose whole body can be removed without any test failing although tests execute them. Descartes implements this "extreme mutation" for Java on top of PIT [14], and Vera-Pérez et al. [15] studied pseudo-tested methods across a set of open-source Java projects. Deng et al. [2] evaluated statement deletion as an economical mutation operator. Maton et al. [8] applied extreme mutation at statement level to 27 Java projects and reported that 48% of pseudo-tested statements lie outside pseudo-tested methods. If that holds for Python, a killed whole-function mutant says little about the function's individual effects.

Test-suite reduction. Reduction techniques remove tests while preserving a requirement such as coverage. Shi et al. [13] showed that coverage-preserving reduction can lose fault-detection ability, measured with mutants. The audit avoids automatic reduction, but it must still decide what evidence a "candidate removal" needs.

Flakiness. Luo et al. [7] categorised the root causes of flaky tests, and Gruber et al. [3] studied them in Python, finding order dependence to be a major category. Both show that intermittent failures can be rare per run, which bounds what a small number of reruns can show.

Mutation at scale. Petrović and Ivanković [11] describe diff-based, filtered mutation analysis at Google, where cost and reviewer attention, not raw mutation scores, decide what is useful. This motivates the cost-per-finding framing of H2.

Tools. We use coverage.py dynamic contexts [1] through pytest-cov to attribute lines to tests, and the operator generator of mutmut 3.8.0 [4] for classical mutants.

3 Method

3.1 Corpus

We chose eight mature, pure-Python projects with pytest suites that run in minutes: attrs, more-itertools, python-dateutil, toolz, marshmallow, click, tomlkit and cachetools. They cover class generation, iterator utilities, parsing and time zones, functional utilities, schema validation, command-line handling, a stateful TOML document model, and caching. Each is pinned to the commit of its latest release tag as of 15 October 2026 (Table 1) and installed into its own virtual environment on CPython 3.12 with pinned test tooling (pytest 9.1.1, pytest-cov 7.1.0, coverage 7.16.2, pytest-randomly 3.16.0). Every run disables pytest-randomly except the random-order runs of H6, and projects that use Hypothesis run with a fixed seed.

Three pristine baseline runs define the scope. A test is in scope only if it passed in all three. Tests that failed in every run (41 in total, all in dateutil: its bundled time-zone archive is generated at release time and absent from a git checkout, and the resulting I/O warning becomes an error under the project's warnings-as-errors filter) or whose outcome or identifier varied (6, all in marshmallow, whose parameter IDs embed the current time) are excluded. No project met the pre-registered exclusion rule.

3.2 Instrument

All experiments use the runner that ships with the plugin. It executes pytest in the project's own interpreter with a small plugin that records each test's setup, call and teardown outcome and duration.

Per-test coverage. pytest-cov's `--cov-context=test` labels each executed line with the test and phase that ran it. We use the *run* phase as a test's coverage set $C(t)$; setup, teardown and import-time lines are kept separately. Coverage is measured once per project and stored as raw data.

Project	Version	Tests	In scope	Suite (s)	Classical	Deletion
attrs	26.1.0	1,383	1,377	4.30	3,918	344
more-itertools	v11.2.1	765	765	8.44	4,439	298
dateutil	2.9.0.post0	2,090	1,985	2.96	7,738	430
toolz	1.2.0	188	187	0.60	1,501	159
marshmallow	4.3.1	1,192	1,186	1.20	2,794	358
click	8.5.0	2,016	1,990	3.46	7,850	1,019
tomlkit	0.15.1	1,051	1,051	0.87	5,381	818
cachetools	v7.2.1	338	338	4.42	1,531	223

Table 1: Corpus at the pinned release commits. *In scope*: tests that passed in all three baseline runs. Mutant columns give the size of each eligible population.

Mutant populations. The *classical* population is every mutant produced by mutmut 3.8.0's operator generator (arithmetic, comparison, constant, argument and string operators). We call that generator directly and run each mutant ourselves, because mutmut's runner stops at the first failing test, which would leave kill sets incomplete. The *deletion* population is ours: whole-function body replacement with `return None` plus constants matching the return annotation (for example `0` and `1` for `int`, `[]` for `list`), and replacement of each single call statement with `pass`. Mutants are spliced line by line so every line keeps its number. Both populations use the same eligibility rule as mutmut: top-level functions and methods of top-level classes, undecorated except for `staticmethod` and `classmethod`, excluding async functions. Body replacement also skips generators and trivial bodies (`pass`, `...`, `raise NotImplementedError`).

Execution and statuses. Each mutant runs alone in a worker's private copy of the project, with the copy's source placed ahead of the editable install on the import path. Unless stated otherwise, the tests run are the in-scope tests whose coverage includes a mutated line. Statuses are: *killed* (a test that passed in all baseline runs fails or errors), *survived* (valid, activated, no killer), *uncovered* (no in-scope test covers it; not run), *not activated* (tests ran but the mutated line never executed), *invalid* (compile or collection/import error), *timeout* (over 10× the expected duration plus 10 s) and *infrastructure failure*.

Activation check. A mutant that never executes cannot be detected, so counting it as a survivor would overstate test weakness. Our plugin registers a `sys.monitoring` line callback when it loads, before conftest files import production code, and records whether the first line of the mutated region ran. The mutant source is not changed. On a known-answer fixture this check correctly separates a mocked-out method (not activated) from an unasserted one (survived).

3.3 Hypotheses and analysis

The pre-registration gives full definitions. In summary:

H1 Among test pairs with coverage Jaccard ≥ 0.9 (union ≥ 5 lines; up to 50 pairs per project), a non-trivial share are *mutually non-dominated*: each kills a mutant the other misses. Up to 40 mutants on lines either test covers run against both tests, recording each outcome. A comparison stratum uses pairs with Jaccard in $[0.5, 0.9)$. Supported if the pooled share's lower 95% bound exceeds 10%.

H2 Under equal 20-minute wall-clock budgets with 8 workers, deletion mutants yield more activated survivors per CPU-minute than classical mutants. Supported if the deletion/classical ratio exceeds 1 in a majority of projects and its pooled lower bound exceeds 1.

H3 A non-trivial share of call-deletion survivors lie inside functions whose every body replacement was killed. All body mutants run first; then up to 300 sampled call deletions per project. Supported if the pooled lower bound exceeds 10%.

H4 In test files with 10 to 60 tests (up to 3 per project, up to 150 mutants each, full kill matrix), removing all leave-one-out-redundant tests together loses kills in more than 20% of files (lower bound).

H5 Per-test contexts slow suites by more than 20% (pooled lower bound of the median ratio above 1.2), from 10 interleaved blocks of no coverage, plain coverage and contexts.

H6 Descriptive: intermittent tests across 30 file-order and 30 random-order runs, and whether a 3-run check would have seen them.

Project	Pairs	Non-dominated	95% CI	One-way	Identical
attrs	50	11 (22%)	13–35%	21	18
more-itertools	50	10 (20%)	11–33%	21	19
dateutil	50	27 (54%)	40–67%	15	8
toolz	50	25 (50%)	37–63%	15	10
marshmallow	50	9 (18%)	10–31%	24	17
click	50	29 (58%)	44–71%	17	4
tomlkit	50	5 (10%)	4–21%	21	24
cachetools	50	24 (48%)	35–61%	16	10

Table 2: H1: test pairs with coverage Jaccard ≥ 0.9. *Non-dominated*: each test kills a mutant the other misses. *One-way*: one kill set strictly contains the other. *Identical*: equal kill sets (including both empty). Per-project intervals are Wilson 95%.

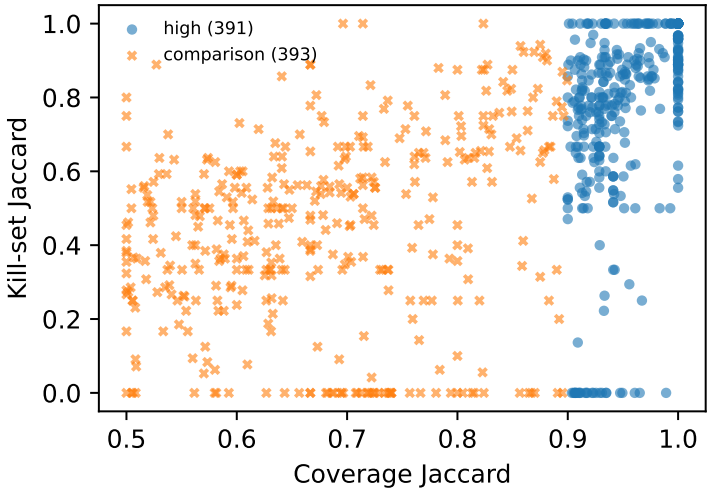


Figure 1: H1: coverage Jaccard against kill-set Jaccard for sampled pairs with at least one kill.

Proportions within a project use Wilson intervals. Pooled shares and ratios use a percentile bootstrap with 10,000 resamples of whole projects, so a project with many pairs or mutants cannot dominate the interval by itself. H5 pools per-project median ratios with a geometric mean.

All tables, figures and summary statistics in this paper are generated from the raw records by the analysis scripts. A few descriptive counts in the text, such as the number of runs in a design or a value read from a generated table, are written out.

4 Results

4.1 H1: coverage overlap and redundancy

Every project had pairs above the 0.9 threshold, so each contributed 50 sampled pairs (400 in total). Among them, 35.0% were mutually non-dominated: each test killed at least one mutant the other did not (95% CI 22.2% to 47.5%, clustered by project). H1 is **supported**. A further 37.5% of pairs were one-way: one test's kill set strictly contained the other's, so only one of the two tests could be removed without losing kills, and the coverage score does not say which. Only the remaining pairs had identical kill sets. Overall, 72.5% of high-overlap pairs differed in what they detect.

The share varied widely between projects (Table 2), from 10.0% in tomlkit to 58.0% in click. Pairs of parametrized cases of the same test function were non-dominated less often (20.2% of 114) than pairs of distinct functions (40.9% of 286), but both are far from zero.

Across both strata, coverage Jaccard and kill-set Jaccard are correlated (Spearman $\rho = 0.69$, $n = 784$ pairs with at least one kill; Figure 1). Overlap is therefore a reasonable signal for *where to look*, which is how the audit uses it, but at the top of the scale it does not reliably separate redundant pairs from complementary ones.

Project	Classical surv.	Classical CPU min	Deletion surv.	Deletion CPU min	Ratio	Deletion exhausted
attrs	300	158.49	9	15.89	0.30	yes
cachetools	193	9.86	14	9.94	0.07	yes
click	462	159.36	69	59.31	0.40	yes
dateutil	377	149.87	31	39.86	0.31	yes
marshmallow	318	62.91	14	9.59	0.29	yes
more-itertools	201	155.65	4	22.26	0.14	yes
tomlkit	566	158.63	85	44.40	0.54	yes
toolz	161	14.88	2	1.26	0.15	yes

Table 3: H2: activated, valid survivors and CPU minutes under equal 20-minute wall-clock budgets. *Ratio*: deletion survivors per CPU-minute divided by classical survivors per CPU-minute. *Deletion exhausted*: every eligible deletion mutant ran within the budget.

4.2 H2: is deletion a cheaper way to find untested code?

No. Under equal 20-minute wall-clock budgets, deletion mutants yielded 0.38 times as many activated survivors per CPU-minute as mutmut's classical operators (95% CI 0.24 to 0.48). The ratio was above 1 in 0 of 8 projects (Table 3), so H2 is **not supported**. This is the clearest negative result of the study, and the opposite of what we expected from the Java literature, which presents extreme mutation as the economical option [2, 14].

Two measurements explain it. First, deletion mutants survive less often: 6.2% of completed deletion mutants survived, against 12.7% of classical ones. Removing a whole function body, or a call, is usually a large behavioural change that some test notices. Second, a deletion mutant is not cheaper to run. The median CPU cost per mutant is almost the same (0.89 s against 0.86 s), but the mean is higher for deletion (3.33 s against 2.56 s). Deletion mutants are covered by more tests (median scope 18 against 12), break more of them when killed (median 8 failing tests against 5), and time out about twice as often (3.9% against 2.0%); we did not investigate the individual timeouts.

The deletion population is also small. It ran to completion inside the budget in 8 of 8 projects, using 203 CPU-minutes in total against 870 for the classical runs, which ran out of budget in five projects. Completing the whole deletion population costs about a quarter of what the classical budget spent, and gives 228 survivors against 2578.

Survivors per CPU-minute is a measure of yield, not of value. A surviving classical mutant is often a small change of a constant or an operator, a population known to contain many equivalent mutants [10], while a surviving deletion says that a whole function or call can disappear unnoticed. The pre-registered usefulness review was designed to compare the two on that axis. At the time of writing, 0 of the 50 sampled survivors have been labelled, so that comparison is pending (real-gap share: classical n/a, deletion n/a). Blinding the reviewer to strategy turned out to be impractical, since a deletion patch is recognisable on sight.

4.3 H3: killed functions with unchecked calls

We ran every body-replacement mutant of every eligible function (1876 functions), then 1251 sampled call-statement deletions. 1411 functions had every replacement killed; most of the rest could not be assessed because their body mutants were uncovered, not activated or invalid.

Of the 140 activated, valid call-deletion survivors, 71.4% lay inside a fully killed function (95% CI 58.0% to 80.3%). H3 is **supported**. Viewed from the functions' side, 20.3% of the 404 fully killed functions that had at least one valid sampled deletion contained a deletion no test noticed (Table 4).

The share is higher than the 48% reported for Java [8]. The two numbers are not directly comparable. We delete only call statements, not all statements, and our function-level population is restricted by the mutmut eligibility rule. Still, the direction is the same and stronger: in these Python projects, most of the effects that tests fail to check sit inside functions that whole-function mutation would report as well tested.

A random sample of 40 survivors (seed 1) was dominated by four kinds of call: `super().__init__(...)` in exception classes, whose message and position attributes no test checks (six of the 40, all in tomlkit); output side effects such as `flush()` and progress-bar rendering in click; cleanup and invalidation such as `_validation_cache.clear()`, `_reindex()` and a linked-list `unlink()` in a TTL cache; and `sort()` calls whose ordering effect no test observes. Whether each matters is a review question. The audit can only show that no test noticed, which is why it reports these as candidate gaps rather than defects.

Project	Call deletions	Valid	Survived	In fully killed fn	Share
attrs	160	107	8	2	0.25
more-itertools	110	100	4	2	0.50
dateutil	183	142	19	11	0.58
toolz	33	32	2	2	1.00
marshmallow	94	77	9	8	0.89
click	300	252	37	31	0.84
tomlkit	257	233	48	34	0.71
cachetools	114	98	13	10	0.77

Table 4: H3: sampled call-statement deletions. *In fully killed fn*: survivors whose enclosing function had every body replacement killed.

Project	Test file	Tests	Killed	LOO-redundant	Lost by group	Greedy cover	No kills
attrs	test_annotations.py	47	31	44	11	6	0
attrs	test_filters.py	33	21	33	21	4	0
attrs	test_funcs.py	53	41	48	8	11	4
toolz	toolz/test_inspect_args.py	17	92	9	4	9	0
toolz	toolz/test_dicttoolz.py	51	139	48	81	14	0
toolz	toolz/test_itertoolz.py	51	138	18	3	34	1
marshmallow	test_utils.py	20	45	13	1	8	5
marshmallow	test_options.py	10	45	6	1	5	0
marshmallow	test_decorators.py	44	84	41	41	8	0
click	test_info_dict.py	28	90	21	18	9	0
click	test_stream_lifecycle.py	23	82	19	2	5	4
click	test_testing.py	43	89	35	8	11	3
tomlkit	test_parser.py	34	80	28	12	8	0
cachetools	test_func.py	36	112	24	6	15	0
cachetools	test_tlru.py	34	126	25	7	13	0
cachetools	test_cache.py	23	66	19	9	7	0

Table 5: H4: kill matrices for sampled test files. *LOO-redundant*: tests whose removal alone loses no kill. *Lost by group*: kills lost when all of them are removed together. *Greedy cover*: tests a greedy set cover keeps.

4.4 H4: individually removable is not jointly removable

16 test files met the size rule. more-itertools and dateutil contributed none. more-itertools keeps its tests in two files of 145 and 620 tests. dateutil's only file in range, `test_imports.py`, has 27 tests, but only 8 of them record run-phase coverage, which leaves it below the 10-test minimum. tomlkit had one eligible file.

In every file, at least two tests were leave-one-out redundant, so all 16 files were evaluated. Removing the whole leave-one-out-redundant set together lost at least one kill in 16 of 16 files (100.0%, Wilson 95% CI 80.6% to 100.0%). H4 is **supported**. Across those files, group removal lost 233 of 1281 kills (Table 5).

The extreme case is attrs' `test_filters.py`: every one of its 33 tests is individually redundant, because every mutant it kills is killed by at least two tests, yet removing them all loses all 21 kills. Leave-one-out redundancy is a statement about one test given all the others, so it cannot be applied to several tests at once. This is the textbook result for set cover; the data shows it is not a corner case on real suites.

The greedy cover column shows how small a subset would preserve every observed kill: a median of 28.9% of the file's tests (range 12.1% to 66.7%). That number is fitted to this mutant sample. It should be read as an upper bound on what a fast-feedback tier could drop, to be checked on independent mutants and historical bugs, not as a deletion list.

4.5 H5: overhead of per-test coverage contexts

Per-test contexts made suites slower by a pooled factor of 2.48 (95% CI 1.74 to 3.88), so H5 is **supported**. The range is wide (Table 6): 1.07 for cachetools, where four threading tests that each sleep for one second account for most of the suite's time, and 9.50 for more-itertools. Most of the cost comes from coverage measurement itself. Adding contexts on top of plain coverage costs a pooled factor of 1.18, with click the clear exception. Click has 2,016 tests with a median duration of 0.2 ms, so a fixed cost per context switch is a large fraction of each test; we did not measure that cost directly. For more-itertools, plain coverage alone is about nine times slower than an uninstrumented run. One test, `test_primes`, takes 4.4 s of the 8.3 s uninstrumented

Project	None (s)	Coverage (s)	Contexts (s)	Cov/None	Ctx/None	Ctx/Cov
attrs	4.29	7.39	9.64	1.73	2.24	1.30
cachetools	4.42	4.65	4.72	1.05	1.07	1.01
click	3.19	5.28	9.55	1.66	3.00	1.81
dateutil	2.78	5.36	5.84	1.93	2.10	1.09
marshmallow	1.02	1.92	2.21	1.88	2.16	1.15
more-itertools	8.73	79.95	83.18	9.24	9.50	1.03
tomlkit	0.79	1.63	1.95	2.06	2.47	1.20
toolz	0.49	0.89	0.93	1.81	1.88	1.04

Table 6: H5: median wall time over 10 interleaved blocks, and median per-block ratios.

Project	Runs	Tests seen	Intermittent
attrs	60	1,383	0
more-itertools	60	765	0
dateutil	60	2,090	2
toolz	60	188	0
marshmallow	60	1,306	0
click	60	2,016	0
tomlkit	60	1,051	0
cachetools	60	338	0

Table 7: H6: tests with both a pass and a failure across 30 file-order and 30 random-order runs.

suite and calls `is_prime` and `nth_prime` for every prime below 100,000: many short pure-Python calls, each of which the tracer has to follow.

The practical consequence for the audit is that timings must come from uninstrumented runs, and any time estimate derived from a coverage run must be labelled as such.

4.6 H6: intermittent outcomes in repeated runs

Across 60 runs of each of the 8 suites (30 in file order, 30 in random order), 2 tests had both passing and failing outcomes, both in dateutil (Table 8). No test in the other seven projects failed intermittently. For a test that never failed, 60 independent runs bound its per-run failure rate at about 4.9% (95%); smaller rates cannot be excluded.

Both dateutil tests check that `dateutil.zoneinfo` emits a `DeprecationWarning`. In our checkout the bundled time-zone archive is absent (it is generated for releases), and reading it emits an I/O `UserWarning`, which the project's warnings-as-errors filter turns into a failure. The two functions under test, `zoneinfo.gettz` and `zoneinfo.gettz_db_metadata`, share a module-level cache, so only whichever of the two tests runs first reads the archive and fails. In file order that is always `testZoneInfoDeprecated`, which therefore failed all 30 file-order runs and was already excluded by the baseline as a consistent failure. In random order the failure moved between the two tests. `testZoneInfoMetadataDeprecated` passed in every file-order run and failed in 15 of 30 random-order runs.

So 1 of the 2 would have been flagged by a three-run check, and only as a consistent failure. The other is invisible to any number of same-order reruns: 1 test failed only under random ordering. This is a small result, but it is the pattern that matters for the audit. Repeating a suite in the same order measures noise; it does not reveal order dependence, which Gruber et al. [3] report as a leading cause of flakiness in Python. Mature libraries like these appear to have little run-to-run noise, so the audit's flake step should vary order deliberately and report "not observed in n runs" with its bound, never "stable".

5 Threats to validity

Mutants as a proxy. Every detection result is relative to our mutant populations. Mutants are a reasonable but imperfect stand-in for real faults [6]. A pair of tests that are "non-dominated" on our mutants could still be equivalent on the faults that matter to a project, and the reverse. Equivalent mutants inflate survivor counts; we did not review survivors for equivalence except in the H2 sample.

Invalid and unactivated mutants. We count mutants that break import or collection as invalid, not killed. This is conservative for kill rates and removes some mutants from H1 and H4 kill sets entirely. The

Project	Test	Fails	Runs	First failure (run)	In random order
dateutil	testZoneInfoDeprecated	45	60	1	15
dateutil	testZoneInfoMetadataDeprecated	15	60	31	15

Table 8: H6: intermittent tests observed.

activation check uses the first line of the mutated region, so a mutant on a later part of a multi-line expression could be marked activated although that sub-expression never ran. Deletion mutants replace the first line, so this does not affect them.

Test selection by coverage. Mutants run only against tests whose run-phase coverage includes a mutated line. A test that detects a mutant only through import-time or fixture code would be missed, so some survivors may be killable by the wider suite. Tests that pass in one session but would fail alone, or the reverse, could also bias H1 and H4 kill sets, because each mutant's tests share one session.

Sampling. H1 and H4 sample pairs, files and mutants with a fixed seed. Projects with very large high-overlap populations (attrs, dateutil) contribute the same 50 pairs as small ones, by design. The H4 file-size rule excluded two projects entirely. Pooled intervals resample projects, but eight projects is a small number of clusters, so the intervals are approximate.

Corpus. Eight mature libraries are not representative of application code, which typically has more I/O, more mocking and more integration tests. Our results are likely to understate mock shielding and overstate unit-level determinism. We chose release commits, whose suites are expected to pass; the one exception (dateutil) was an environment effect of building from a git checkout.

Platform. All runs used one macOS arm64 machine. A Linux replication of H1, H3 and H5 is prepared as a GitHub Actions workflow (`.github/workflows/study-replication.yml`) but has not been run for this version of the paper. Timing results (H2, H5) are the most platform-sensitive.

Analyst. The study was designed, run and analysed by the plugin's author. The pre-registration, the append-only raw data and generated numbers limit, but do not remove, that bias. The H2 usefulness review is done by the same person.

Deviations. All deviations were decided before the affected data were collected and are listed in the pre-registration: complete kill sets come from one session per mutant rather than one process per test; activation uses runtime line monitoring instead of an inserted sentinel; H3 call deletions are sampled from all functions so that the primary metric has a valid denominator; four projects' first baseline attempts failed for harness reasons and were re-run. One defect was found after collection: toolz keeps its tests inside its package, so H2 and H3 also mutated toolz test files. Those runs were exhaustive and each mutant runs independently, so the analysis drops the test-file mutants, which is equivalent to never generating them; the H3 result does not change because both toolz survivors were in production code.

6 Implications for the Test Suite Auditor

Each result maps to a requirement in the product specification [12].

Overlap is a lead, not a verdict (PRD F4; H1). Supported. High-overlap pairs are worth investigating, since overlap correlates with kill-set similarity, but a third of them are complementary. The audit reports overlap only as an *inferred* "investigate" finding and never as a removal candidate.

Fine-grained ablation under killed parents (PRD F6a; H3). Supported, strongly. Most unchecked calls sit inside functions whose whole-body mutants are all killed. The audit must sample call and statement deletions beneath killed functions as well as surviving ones, and must never infer a child result from a parent kill. Deterministic call deletion finds these directly and its population is small enough to run in full; it stays in the MVP.

Group validation of removals (PRD F7; H4). Supported, strongly. Every evaluated file lost kills when its leave-one-out-redundant tests were removed together. Leave-one-out results alone cannot authorise combined removal, and greedy covers fitted to one mutant sample need a holdout check before they are shown as more than an upper bound.

Profile without coverage (PRD F2; H5). Supported. Coverage, and contexts on top of it, change run time by large and project-specific factors. Timings come from uninstrumented runs; coverage runs are labelled as such.

Vary order when looking for flakiness (PRD F3; H6). The only intermittent tests we found were order-dependent: same-order repetition shows one as a consistent failure and never shows the other failing. The audit's repeat runs should include order variation by default where a random-order plugin is available, and report "not observed in n runs" with its bound.

Stable test identifiers (PRD F1). Not a hypothesis, but a finding of the baseline step: marshmallow parametrizes tests with the current time, so their IDs change on every run and they cannot be tracked across runs. The audit now reports such tests as a separate finding.

Deletion is not a cheap substitute for classical mutation (PRD F5, F6a; H2). Not supported, and the plugin changes accordingly. The PRD's feasibility plan treated deterministic deletion as an economical first pass. On these projects it yields fewer survivors per CPU-minute than mutmut's operators, not more. The audit keeps both populations and reports them separately, as the PRD already requires, but no longer presents deletion as the cheaper option. Deletion stays in the MVP for a different reason, given by H3: it is the direct way to find effects that no test checks, and its whole population is small enough to run completely within a deep-mode budget, which makes "every replacement was tried" labels possible. Whether its survivors are more often worth acting on is the open question the pending review addresses.

References

[1] Ned Batchelder. Coverage.py: measurement contexts, version 7.16. `https://coverage.readthedocs.io/en/latest/contexts.html`, 2026.

[2] Lin Deng, Jeff Offutt, and Nan Li. Empirical evaluation of the statement deletion mutation operator. In *Proceedings of the IEEE Sixth International Conference on Software Testing, Verification and Validation*, pages 84–93, 2013. doi: 10.1109/ICST.2013.20.

[3] Martin Gruber, Stephan Lukasczyk, Florian Krois, and Gordon Fraser. An empirical study of flaky tests in Python. In *Proceedings of the 14th IEEE Conference on Software Testing, Verification and Validation*, pages 148–158, 2021. doi: 10.1109/ICST49551.2021.00026.

[4] Anders Hovmöller. mutmut: Python mutation tester, version 3.8.0. `https://github.com/boxed/mutmut`, 2026.

[5] Laura Inozemtseva and Reid Holmes. Coverage is not strongly correlated with test suite effectiveness. In *Proceedings of the 36th International Conference on Software Engineering*, pages 435–445, 2014. doi: 10.1145/2568225.2568271.

[6] René Just, Darioush Jalali, Laura Inozemtseva, Michael D. Ernst, Reid Holmes, and Gordon Fraser. Are mutants a valid substitute for real faults in software testing? In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, pages 654–665, 2014. doi: 10.1145/2635868.2635929.

[7] Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov. An empirical analysis of flaky tests. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, pages 643–653, 2014. doi: 10.1145/2635868.2635920.

[8] Megan Maton, Gregory M. Kapfhammer, and Phil McMinn. Exploring pseudo-testedness: Empirically evaluating extreme mutation testing at the statement level. In *Proceedings of the IEEE International Conference on Software Maintenance and Evolution*, 2024.

[9] Rainer Niedermayr, Elmar Juergens, and Stefan Wagner. Will my tests tell me if I break this code? In *Proceedings of the International Workshop on Continuous Software Evolution and Delivery*, 2016. doi: 10.1145/2896941.2896944.

[10] Mike Papadakis, Marinos Kintis, Jie Zhang, Yue Jia, Yves Le Traon, and Mark Harman. Mutation testing advances: An analysis and survey. *Advances in Computers*, 112:275–378, 2019. doi: 10.1016/bs.adcom.2018.03.015.

[11] Goran Petrović and Marko Ivanković. State of mutation testing at Google. In *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice*, pages 163–171, 2018. doi: 10.1145/3183519.3183521.

[12] Tim Richardson. Product requirements: Test suite auditor, version 0.2. `https://github.com/timini/test-suite-auditor/blob/main/docs/PRD.md`, 2026.

[13] August Shi, Alex Gyori, Milos Gligoric, Andrey Zaytsev, and Darko Marinov. Balancing trade-offs in test-suite reduction. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, pages 246–256, 2014. doi: 10.1145/2635868.2635921.

[14] Oscar Luis Vera-Pérez, Martin Monperrus, and Benoit Baudry. Descartes: A PITest engine to detect pseudo-tested methods. In *Proceedings of the 33rd IEEE/ACM International Conference on Automated Software Engineering, Tool Demonstrations*, 2018. doi: 10.1145/3238147.3240474.

[15] Oscar Luis Vera-Pérez, Benjamin Danglot, Martin Monperrus, and Benoit Baudry. A comprehensive study of pseudo-tested methods. *Empirical Software Engineering*, 24:1195–1225, 2019. doi: 10.1007/s10664-018-9653-2.

A Reproducibility

Everything needed to rerun the study is in the `study/` directory of the repository:

- `PREREGISTRATION.md`: hypotheses, metrics, decision rules and deviations;
- `corpus.toml`: repository URLs, release tags, commit SHAs, install and pytest arguments;
- `harness/`: cloning at the pinned commit, environment setup, raw-record writer;
- `experiments/`: one script per experiment (`e0_baseline` to `e6_flaky`);
- `data/raw/`: append-only JSONL records (gzipped once a run has ended), one file per run, each carrying the project commit, runner commit, platform, Python version, environment hash (of `pip freeze`) and seed; every mutant result stores its exact patch;
- `analysis/`: scripts that produce `data/derived/`, the tables, the figures and `paper/generated/macros.tex`.

The full pipeline is:

```
cd study
make corpus      # clone and install the eight projects
make run         # E0 baseline, then H5, H1, H3, H4, H6, H2
make compress    # gzip the raw files of finished runs
make analyse     # derived CSVs, tables, figures, macros
make paper       # this PDF
```

H5 and H2 measure time, so run them with nothing else running on the machine. The primary runs used an Apple-silicon Mac (macOS, arm64) with 10 cores and 16 GB of memory, and 8 mutant workers.